SOFTWARE ARCHITECTURE · ARCHITECTURAL KATA

ClashArena

An online esports tournament platform — the domain kata and the architecture characteristics derived from it.

Course	Software Architectures
Deliverable	Kata — an IT system to manage sport tournaments
Domain	Online esports (individual 1v1 and team 5v5 titles)
Date	August 2026

Team

Team member	Kata section owner
Daniil Glazunov	Identity, integrity & security; result ingestion & reliability
Shattyk Kuziyeva	Tournament format & scalability/elasticity; users & public read surfaces

Contents

1 About this kata

A **kata** is a short, high-level, domain-targeted description of an IT system, used as the starting point for architectural thinking. Following the course format, a kata is composed of four parts — **Description**, **Users**, **Requirements** and **Additional context** — from which the architect then derives the system's *architecture characteristics* (the “-ilities”).

This document states the kata for our chosen domain, an online esports tournament platform, and then derives and prioritises its architecture characteristics — the next step of the method. Choosing an architecture style and designing the system come later and are out of scope here.

2 The kata

2.1 Description

ClashArena is an online platform that runs competitive video-game tournaments end to end for a set of supported titles — one individual **1v1** title and one **5v5** team title. Organisers create tournaments; players and teams register and check in; the platform seeds them and generates qualifiers, group stages and elimination brackets; matches are played on external game servers that report their results back; the platform validates each result, resolves disputes, advances winners, updates player and team ratings, and drives public leaderboards, tournament and match pages, and live-stream overlays in near real time.

The defining property of the domain is its **load shape**. Between events the platform is quiet, but during marquee events (majors, finals) the number of active competitors — and, far more sharply, the number of spectators — rises by orders of magnitude for a few hours before falling back to baseline.

2.2 Users

Types of users:

- **Players** — individual competitors with profiles and ratings.
- **Teams and team captains** — rosters, player roles, team registration.
- **Organisers / tournament admins** — create and operate tournaments, seeding, scheduling.
- **Referees / moderators** — verify results, resolve disputes, handle no-shows and forfeits.
- **Spectators / fans** — the largest and most volatile group; read brackets, leaderboards and match pages and watch live streams.
- **Casters / broadcasters** — consume live data for stream overlays.
- **Platform administrators** — accounts, bans, anti-cheat and integrity.

Expected numbers (these make the two load characteristics concrete):

- **Registered accounts**: expected to grow from about **50,000 to 500,000 over ~24 months**.
- **Concurrent competitors** in a typical weekly cup: a few thousand.
- **Spectators during a major final**: short bursts of about **100,000–500,000** concurrent read / stream clients for a few hours, returning to a ~5,000 baseline afterwards.
- **Match results ingested**: several thousand per hour during peak event windows.

2.3 Requirements

Domain-level requirements expected by organisers, players and moderators:

1. Account registration and authentication; player profiles; team creation with rosters and roles.
2. Organisers create tournaments with a format (single / double elimination, round-robin group stage, Swiss qualifier), rules, schedule and prize information.
3. Registration and check-in windows, seeding (by rating or manual) and waitlists.

4 Automatic generation of groups and brackets, and scheduling of matches into time slots.
5 Match lifecycle: external game servers or organisers report results; the platform validates them, resolves disputes, advances winners and handles no-shows and forfeits.
6 Near-real-time standings, brackets and player / team rating updates.
7 Public, read-heavy surfaces: leaderboards, tournament pages, match pages and live-stream overlay feeds.
8 Notifications for match start, results and schedule changes.
9 Anti-cheat and integrity checks with moderator tooling (bans, result reversals, audit trail).
10 Analytics and reports for organisers (participation, viewership, match statistics).

2.4 Additional context

- **Spiky, event-driven load.** Traffic is low between events and spikes by orders of magnitude during majors — spectator (read) traffic most of all. The competitive write path must stay responsive while spectator traffic peaks.
- **Result ingestion is a continuous stream.** Results arrive from many external game servers and must be received, normalised, validated and published with **no loss and no double-counting**.
- **Integrity is critical.** Results and rating changes must be tamper-resistant and auditable, and disputes must be traceable; cheating undermines the whole product.
- **Global audience.** Competitors and spectators are worldwide, so low latency on read surfaces matters.
- **Third-party integrations.** Game servers, streaming platforms, and payments for entry fees and prizes.
- **Independent evolution.** Distinct domains (identity, tournaments, result ingestion, ratings, streaming, moderation) are built by different team members and should evolve and deploy independently.
- **Reproducible and cloud-ready.** The platform must build and run in containers, locally and in the cloud, and scale on demand.
- **Availability during events is non-negotiable.** An outage during a broadcast final is a reputational disaster.

> **Forward pointer — the result-ingestion stream**
>
> The stream of match results maps naturally onto a **producer** → **transformer** → **tester** → **consumer** pipeline: game servers *produce* raw results, a *transformer* normalises them (server payload → match / player), a *tester* validates and de-duplicates, and a *consumer* publishes standings, ratings and overlays. This shapes the reliability and fault-tolerance targets below.

3 Architecture characteristics derived from the kata

3.1 Scalability vs elasticity in this domain

The domain exhibits both of the course's headline operational characteristics, and telling them apart is the crux of this kata. **Scalability** is the platform's ability to keep performing as the registered base and event sizes grow steadily over months (left, below). **Elasticity** is its ability to absorb sudden bursts of users — chiefly spectators — within a single event and then release the capacity afterwards (right). They demand different mechanisms: scalability is a capacity-planning and horizontal-growth concern, whereas elasticity is an autoscale-and-release concern driven by short, sharp spikes.

Figure 1 — The ClashArena load profile: steady growth (scalability) vs event-day bursts (elasticity).

3.2 Explicit characteristics

Characteristics stated, directly or indirectly, by the Requirements and Users sections:

Characteristic	Where it comes from in the kata
Elasticity	Additional context (event bursts) + Users (spectator spikes of 100k–500k).
Scalability	Users (growth to ~500k accounts) + Requirements 2/4 (ever-larger events).
Performance / responsiveness	Requirements 6/7 (near-real-time standings; read-heavy public surfaces).
Reliability	Additional context (ingestion with no loss and no double-count).
Fault tolerance	Additional context (external game-server and streaming feeds may fail).
Availability	Additional context (non-negotiable during scheduled events).
Security / integrity	Requirement 9 + Additional context (tamper-resistant, auditable results).
Deployability / modularity / evolvability	Additional context (independent domain ownership and updates).
Testability	Requirement 5 (the validate / dispute logic must be verifiable).

3.3 Implicit characteristics

Some essential characteristics are never written down by stakeholders but must be made explicit — exactly the point the course makes about security and availability. In this domain we surface three and treat them as first-class even though the kata states them only indirectly: **security** (nobody requests it, but a tamper-able results platform is worthless), **availability** (spectators simply assume the site is up during a final), and **data integrity / auditability** of results and ratings.

3.4 Driving characteristics and measurable targets

Every characteristic adds complexity and cost, so we keep a minimal **driving set** and give each a measurable target. Targets are set as concrete numbers to be refined with stakeholders and tuned through the DevOps feedback loop (SMART — “set a number, iterate”).

> **Driving set (the few that matter most)**
>
> **1. Elasticity 2. Scalability 3. Reliability & fault tolerance 4. Security & integrity.**
> Supporting: performance / responsiveness, deployability / modularity, testability.

Characteristic	Measurable target (initial)
Elasticity	Absorb a spectator burst from ~5k to ~300k concurrent read clients within ~2 min by autoscaling, no manual action, then release capacity.
Scalability	Sustain 10× growth in registered accounts and 5× in concurrent competitors over 24 months with no redesign.
Performance / responsiveness	Leaderboard / bracket reads P95 < 300 ms globally; a recorded result appears in standings within ~5 s.
Reliability (ingestion)	Every match result processed exactly once — no loss, no double-count — under peak load.
Availability	≥ 99.95% during scheduled event windows.
Fault tolerance	A failed game-server or streaming feed must not block result recording or take down read surfaces (graceful degradation).
Security / integrity	Results and rating changes authenticated, authorised (RBAC) and auditable; sensitive data encrypted in transit and at rest.
Testability	The ingest → validate → publish path is covered by automated tests, including fault injection, in CI.

3.5 Characteristics framework mapping

Placing the selected characteristics in the course's framework — *what is needed* to build and measure them (code, artifact, deployment) and *what they concern* (speed, robustness, structure):

Characteristic	What is needed	What it concerns
Elasticity	Deployment	Speed
Scalability	Deployment	Speed
Performance / responsiveness	Artifact	Speed
Reliability	Artifact, Deployment	Robustness
Fault tolerance	Deployment	Robustness
Deployability	Artifact	Structure
Modularity	Code	Structure
Testability	Artifact	Robustness
Simplicity	Code	Structure

Security, **availability** and **integrity** are *cross-cutting* characteristics: they do not sit cleanly in the structural / operational split and instead constrain the whole system.

4 Next step

From this prioritised set of characteristics the team will choose an architecture style and design the system — the phase that follows this kata in the course. The event-driven, burst-heavy, integrity-critical nature of the domain, and in particular the streaming result-ingestion pipeline together with the elasticity target, will be the main forces shaping that decision.